

# 工程师如何驾驭智能体

从完整交付闭环、Codex App 到开放代理栈

独立学习版 5.6

演讲者：Alexander Embiricos、Romain Huet (OpenAI) <sup>1</sup>



**来源活动：** WF2026 · Software Factories & Keynotes

**主题范围：** 原视频 00:28:37–00:48:09；语义收束延伸至 00:48:12

**主题类型：** OpenAI keynote

**视频发布日期：** 2026 年 7 月 1 日

**视频入口：** <https://www.youtube.com/watch?v=htM02KMNZnk&t=1717s>

**文本依据：** 英文原始字幕用于核对陈述、术语与绝对时间。

**封面范围：** 封面是整场 WF2026 原视频的官方全局封面；本文只覆盖主题 03。

**资料定位：** 正文提供完整背景、机制、实例、限制与实践清单；原视频用于核对现场表达与画面。

<sup>1</sup>姓名依据原视频开场身份页核定；该页显示 Alexander Embiricos 为 Head of Enterprise Product, Romain Huet 为 Head of Developer Experience。

# 目录

|                               |           |
|-------------------------------|-----------|
| 导 读                           | ii        |
| <b>1 工程师仍是主角：从写代码回到解决问题</b>   | <b>1</b>  |
| 1.1 工程实践的核心产出                 | 1         |
| 1.2 自动化扩大后仍需保留的判断             | 2         |
| 1.3 公开建造未来的类比边界               | 3         |
| 1.4 本章小结                      | 3         |
| <b>2 从补全到委派：能力阶梯与完整交付闭环</b>   | <b>3</b>  |
| 2.1 五级能力阶梯                    | 4         |
| 2.2 构建一测试闭环如何工作               | 5         |
| 2.3 从编码扩展到完整交付链               | 5         |
| 2.4 闭环成立的隐藏依赖                 | 7         |
| 2.5 本章小结                      | 7         |
| <b>3 赋能型协作：从目标沟通到细节控制</b>     | <b>8</b>  |
| 3.1 两种可切换的协作状态                | 8         |
| 3.2 理解、控制与责任来自哪些操作            | 9         |
| 3.3 Codex App 的下钻路径           | 10        |
| 3.4 CLI、IDE 与 Codex App 的入口选择 | 12        |
| 3.5 本章小结                      | 12        |
| <b>4 开放代理栈：从模型到插件的六层结构</b>    | <b>12</b> |
| 4.1 六层结构与职责边界                 | 13        |
| 4.2 多步执行怎样持续运行                | 14        |
| 4.3 长时任务怎样保留关键状态              | 15        |
| 4.4 App Server 如何连接客户端        | 15        |
| 4.5 开放组件怎样形成生态反馈              | 16        |
| 4.6 同源构建主张的可信边界               | 18        |
| 4.7 本章小结                      | 19        |
| <b>5 智能体价值：成本、速度与任务运行位置</b>   | <b>20</b> |
| 5.1 怎样评价完整任务的价值               | 20        |
| 5.2 成本与高能力模型的现场比较             | 20        |
| 5.3 速度如何扩大方案搜索                | 21        |
| 5.4 多任务怎样避免相互干扰               | 22        |
| 5.5 任务应在哪个环境运行                | 23        |

|                               |           |
|-------------------------------|-----------|
| 5.6 本章小结 . . . . .            | 24        |
| <b>6 总结与延伸：工程师掌控下的智能体工作系统</b> | <b>25</b> |
| 6.1 从目标到验收的落地清单 . . . . .     | 25        |
| 6.2 开放代理栈中的责任分配 . . . . .     | 26        |
| 6.3 演讲者的实质性收束 . . . . .       | 27        |
| 6.4 开放问题与下一步 . . . . .        | 27        |
| 6.5 本章小结 . . . . .            | 28        |

导读

本资料围绕一个核心问题展开：当智能体能够持续修改、运行工具并推进复杂任务时，工程师应当怎样组织工作，才能同时获得更大的执行范围、清晰的过程控制与可负责的交付结果？六章依次从责任主体、能力反馈、协作界面、系统结构、价值条件和团队落地回答这个问题。

核心回答

智能体扩大了可自动执行的步骤，工程师仍需定义目标与边界、选择验证信号、控制权限与运行环境、判断证据，并对合并、部署和真实影响负责。Codex App（Codex 应用）与开放技术栈提供协作和扩展入口；可靠交付取决于完整任务的成本、速度、环境、验收与责任是否共同成立。

表 1: 阅读路径：六章分别解决的关键问题

| 章节  | 关注对象            | 读者应获得的判断能力                                  |
|-----|-----------------|---------------------------------------------|
| 第一章 | 工程师的价值与责任       | 判断代码生成能力扩大后，哪些问题定义、设计取舍和最终责任仍需明确主体承担        |
| 第二章 | 从候选代码到完整任务      | 区分局部辅助、反馈纠错与持续执行，并识别可靠交付所需的环境和验证条件          |
| 第三章 | Codex App 的协作方式 | 决定何时从目标层沟通，何时进入具体产物检查、修改和验收                 |
| 第四章 | 六层开放技术栈         | 说明模型、接口、Codex Harness、客户端控制与插件的职责、开放方式和可信边界 |
| 第五章 | 成本、速度与执行位置      | 区分基准分数、价格、生成吞吐量和端到端任务价值，并选择合适运行环境           |
| 第六章 | 团队落地与独立验收       | 把目标、权限、观察、验证、回退和责任组织成可执行检查清单                |

## 1 工程师仍是主角：从写代码回到解决问题

### 本章结论

工程实践（engineering）的核心产出是可用、可验证、由明确主体负责的问题解决结果。代码承担实现功能，科学、设计、品味、判断与想象力共同决定应该解决什么、采用哪条路径、何时算完成，以及结果能否进入真实环境。智能体扩大执行能力后，工程师的责任进一步集中到目标、约束、取舍、验收与后果承担。

### 1.1 工程实践的核心产出

把工程实践缩减为“编写代码”，会同时遗漏上游的问题定义和下游的结果责任。真实项目通常从一个不完整的需要开始：用户表达的是现象，组织给出的是目标，仓库保存的是历史选择，运行环境还带着成本、权限和合规边界。工程师需要把这些材料转化为可执行的问题，并判断最终产物是否真正改善了原先的处境。

演讲者把这项工作拆成五种相互补充的能力。它们共同解释了代码生成能力增强之后，工程判断为何仍是交付的控制面。

1. **科学**提供关于因果、可行性和证据的约束，例如一个性能瓶颈是否由数据库往返造成，某项改动能否被重复测量。
2. **设计**安排系统、界面和流程之间的关系，使解决方案适合真实使用场景，并处理易用性、复杂度、成本与可维护性的取舍。
3. **品味**识别局部正确之外的整体质量，例如接口是否一致、概念是否过多、实现是否给未来留下清晰的修改空间。
4. **判断**在信息不完整时作出有责任边界的选择，包括哪些风险可以接受、哪些证据仍然不足、何时需要停止自动执行并升级给人工。
5. **想象力**提出当前材料没有直接给出的可能路径，把模糊需要转化为可试验的原型与备选方案。

这五项能力分别回答“事实允许什么”“方案如何组织”“结果是否协调”“此刻应当怎么选”与“还可以尝试什么”。工程结果由这些答案共同塑造。智能体可以帮助检索、实现、运行工具和比较候选方案，目标正当性、关键取舍和生产责任仍需由有授权的人承担。

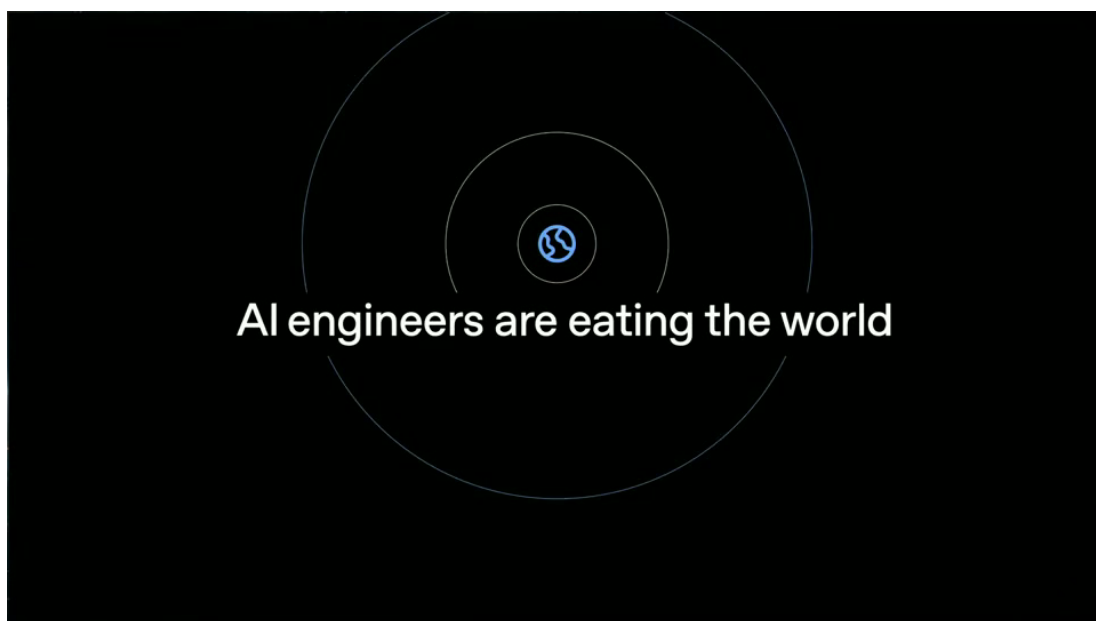


图 1: 演讲以“AI 工程师正在重塑世界”的修辞强调：价值来自把 AI 能力转化为真实可用的工程结果。该页表达演讲者的产业判断，不是就业规模或市场份额统计。<sup>2</sup>

图 1 的句子承担动员和议题转换功能。它可以转化为一项工程要求：当软件与 AI 能力进入更多行业时，工程师需要完成从模型输出到用户价值之间的翻译、集成和验证。画面没有提供行业渗透率、岗位变化或可靠性数据，因此不能支持劳动力市场层面的结论。

## 1.2 自动化扩大后仍需保留的判断

执行能力扩大，会重新分配工程时间。局部语法、常规改动和可重复的工具调用可以更多地交给智能体；工程师因此获得更多机会处理高杠杆判断。这里的“高杠杆”至少包含四个层次。

**问题定义** 把“做一个功能”改写成用户行为、约束和可观察的成功条件，并确认问题值得解决。

**方案选择** 比较多条实现路径的收益、复杂度、迁移成本和失败模式，避免把最先生成的方案当作默认答案。

**证据设计** 决定需要哪些测试、评审、运行观测和用户反馈。测试通过只能证明已检查的条件成立，不能自动覆盖未编码进测试的需求。

**责任验收** 确认改动是否可以合并、发布或回退，记录谁批准了高风险动作，并在失败后承担调查与修复责任。

### 容易混淆的三件事

**生成速度**说明候选产物出现得更快；**工程质量**还取决于需求、架构、验证与运行结果；**责任归属**要求有授权的主体对部署和影响作出决定。三者彼此相关，任何一个指标都不能替代另外两个。

工程师责任集中并不意味着每一步都要亲手完成。有效分工会把可重复执行交给工具，把不可逆决定、模糊目标和高影响验收留在清晰的人工门禁中。随着自动化触及更多环节，门禁的位置、通过条件和升级路径需要更加明确，否则执行范围扩大只会同步扩大错误的传播半径。

<sup>2</sup>原视频：00:30:20--00:30:42。



### 1.3 公开建造未来的类比边界

演讲用世界博览会建立直觉：公开建造会把抽象的未来变成可观看、可进入、可讨论的对象。对于 AI 工程，这个类比的值在于强调原型和实际系统。一个可运行的演示能够暴露接口摩擦、性能瓶颈和用户理解偏差，也能让团队围绕同一对象作出更具体的判断。

#### 类比提供的工程启示

“让未来可见”可以转化为一条实践原则：尽早把想法做成可操作、可观察、可质疑的产物。原型的任务是减少关键不确定性，例如确认用户是否理解交互、工具权限是否足够、端到端路径是否成立。原型仍需声明它验证了什么，以及哪些生产条件尚未被覆盖。

世界博览会展示的是经过策划的可见成果，生产系统还要面对长期运行、维护、权限、数据治理、异常恢复和组织协作。舞台演示能够证明某条路径在特定条件下曾经运行；它无法给出普遍失败率，也无法替代持续监控和独立验收。这个边界使类比保持启发性，同时避免把“看见未来”误读为“已经完成可靠交付”。

因此，“回到工程实践的根本”可以落实为一条责任链：先定义有价值的问题，再给出受约束的方案，随后建立足够的验证证据，最后由明确主体决定交付、回退或继续探索。代码生成位于这条链中，无法单独代表整条链。

### 1.4 本章小结

- 工程实践交付的是可用且可验证的问题解决结果，代码是其中的实现媒介。
- 科学、设计、品味、判断与想象力共同约束目标、方案和质量，智能体执行不能取消这些判断。
- 原型与公开演示能够减少不确定性，其证据范围受场景、权限、验证和长期运行条件限制。
- 工程师持续承担目标、边界、验收与结果责任；执行范围越大，责任门禁越需要清晰。

下一章将把“执行范围扩大”拆成五级能力，说明智能体何时从提供候选代码进入可评审的任务闭环。

## 2 从补全到委派：能力阶梯与完整交付闭环

#### 本章结论

智能体能力跃迁的关键是反馈闭环。系统能够运行改动、观察结果、依据失败修正并重新验证之后，工作单位才会从候选代码扩大为可评审的委派任务。可靠委派还要求目标、环境、权限、验证与人工验收同时成立；生成能力本身不足以构成交付能力。

演讲者用发布节奏解释工程感受为何迅速变化：他把过去约十五个月一次的新模型发布，和会议时点约六周一次的节奏相比较，并称前一周发布了 5.6 系列预览。<sup>3</sup> 这组现场陈述提供了理解后续能力阶梯的时点背景。

<sup>3</sup>原视频：00:31:10–00:31:34。时效与证据范围：约十五个月、约六周和 5.6 系列预览均为演讲者在会议现场的概括，未独立复核完整发布时间线、版本状态或可用范围；这些陈述不代表当前发布频率或当前产品规格，也不能用于预测后续发布节奏。

## 2.1 五级能力阶梯

演讲把开发辅助能力描述为一条不断扩大的连续谱。这条连续谱可以分为五级：代码补全、行内预测、指令式修改、构建—测试闭环（build-and-test loop）与任务委派（task delegation）。五级之间最重要的差异是**系统负责推进到哪里，以及工程师依据什么信号接手判断**。

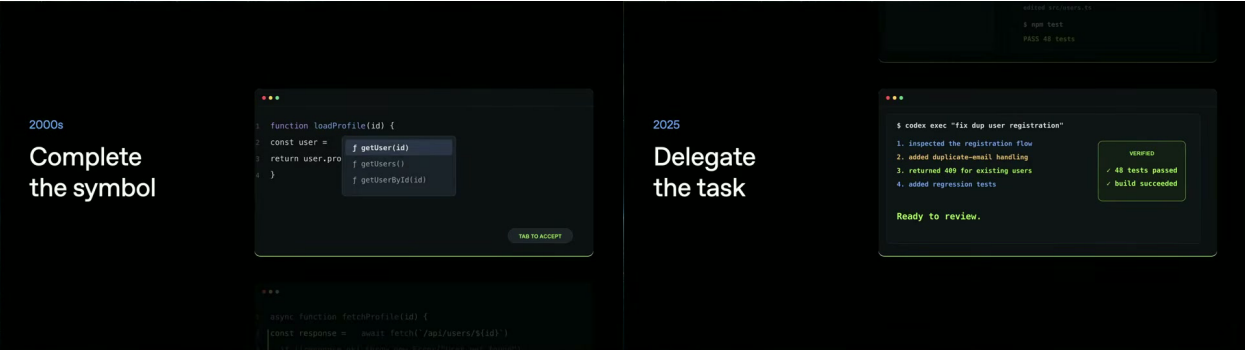


图 2：画面用首尾对照展示工作单位的扩大：左侧系统只补全局部符号，右侧代理检查流程、修改实现、运行验证，并把结果交到“可评审”状态。中间层级由演讲口述与相邻画面补全。<sup>4</sup>

表 2：五级能力阶梯：任务单位、系统动作、验证信号与工程师责任<sup>5</sup>

| 能力层级    | 任务单位             | 系统动作与验证信号                            | 工程师责任                     |
|---------|------------------|--------------------------------------|---------------------------|
| 代码补全    | 一个符号或短代码片段       | 根据局部上下文预测候选；信号主要是编辑器接受与语法可用性         | 判断语义是否正确，确认候选符合当前接口和安全边界  |
| 行内预测    | 一处连续编辑或多行改动      | 结合更大编辑上下文起草候选差异；可观察信号是差异内容与静态提示      | 审查改动范围、遗漏分支和与仓库约定的冲突      |
| 指令式修改   | 一个用自然语言描述的局部目标   | 解析目标并修改一个或多个文件；通常仍由使用者启动构建与测试        | 提供足够上下文，执行验证，拒绝偏离目标的改动    |
| 构建—测试闭环 | 一个具有可执行验证条件的实现目标 | 修改、构建、运行测试、读取失败、修正并复验；信号来自构建、测试和其他检查 | 设计可信验证，限制工具与权限，设置停止和升级条件  |
| 任务委派    | 一个可持续推进到评审状态的任务  | 接收目标、上下文、环境与验收条件，分步调用工具并汇总证据         | 定义完成标准，核对过程与证据，批准合并、部署或回退 |

表 2 展示的是职责边界。层级越高，代理需要的上下文、工具权限和运行时间越多，失败影响范围也随之扩大。低层候选容易逐行检查；高层任务必须依靠结构化日志、差异、测试报告和人工评审保持可见性。

<sup>4</sup>原视频：00:31:47--00:32:12；合成图的两处画面点位为 00:31:47 与 00:32:11。  
<sup>5</sup>证据范围：年份标签与舞台案例表达演讲者对能力演进的概括；表中的五级用于区分任务单位、验证信号和责任边界，不表示所有仓库具备相同成熟度，也不提供各层的普遍成功率。模型与产品版本属于本次会议的现场快照，不能据此断言当前可用性或未来发布节奏。



## 2.2 构建—测试闭环如何工作

构建—测试闭环把一次生成变成一个可观察的纠错过程。它至少需要五类输入：明确的改动目标、可访问的代码与依赖、能够运行的环境、允许调用的工具，以及能区分进展与失败的验证信号。缺少其中任一项，系统都可能产生看似完整、实际无法确认的结果。

一个最小循环按以下顺序运行：

1. **理解目标与现状**：读取相关代码、配置、约定和已有测试，明确允许修改的范围。
2. **形成小步计划**：选择可验证的最小改动，记录预期行为与失败时的回退点。
3. **执行改动**：编辑实现和必要的测试，保持差异可审查，并避免顺手扩大需求范围。
4. **运行验证**：执行构建、单元测试、集成检查或其他任务相关工具，保存命令、退出状态和关键输出。
5. **解释反馈**：区分实现错误、测试错误、环境错误和依赖故障；不同原因需要不同动作。
6. **修正并复验**：根据证据调整实现，重新运行受影响检查，直到达到完成标准或触发停止条件。

### 闭环成立的最低条件

可靠闭环需要**可复现环境、受控权限、可解释反馈、覆盖目标的验证信号、有限重试与明确停止条件**。测试通过只说明已编码的检查通过；需求理解、测试充分性、安全影响和真实用户价值仍需独立判断。

演讲用两段舞台经历说明这一变化。演讲者回顾 2024 年 DevDay 的预览阶段 o1：系统可以从零起草迷你无人机界面，当时无法运行代码或验证自身工作；他又称 2025 年已有更强信心让系统测试其工作，并将其用于现场相机与灯光控制。这些经历说明了“有无反馈闭环”的产品差异。<sup>6</sup>

工程师在闭环中的关键任务是设计**验证依据**。若测试只检查旧行为，代理可能在全部测试通过的同时遗漏新需求；若环境与生产差异过大，本地成功也可能在部署后失效；若失败输出含糊，代理可能重复修补表面症状。高质量闭环因此需要覆盖功能、数据、性能、安全和回归风险的分层检查，并为超时、重复失败和范围漂移设置人工升级。

## 2.3 从编码扩展到完整交付链

构建—测试闭环解决实现阶段的纠错，完整交付还需要连接编码前后的工作。演讲画面把这条链概括为计划、构建、评审与部署。其核心含义是：代理开始接触“为什么做、怎样实现、如何验收、怎样进入真实环境”四类连续问题。

<sup>6</sup>证据范围：两段经历来自特定舞台环境，只能作为能力变化的案例，不能用于推导普遍失败率或安全等级。

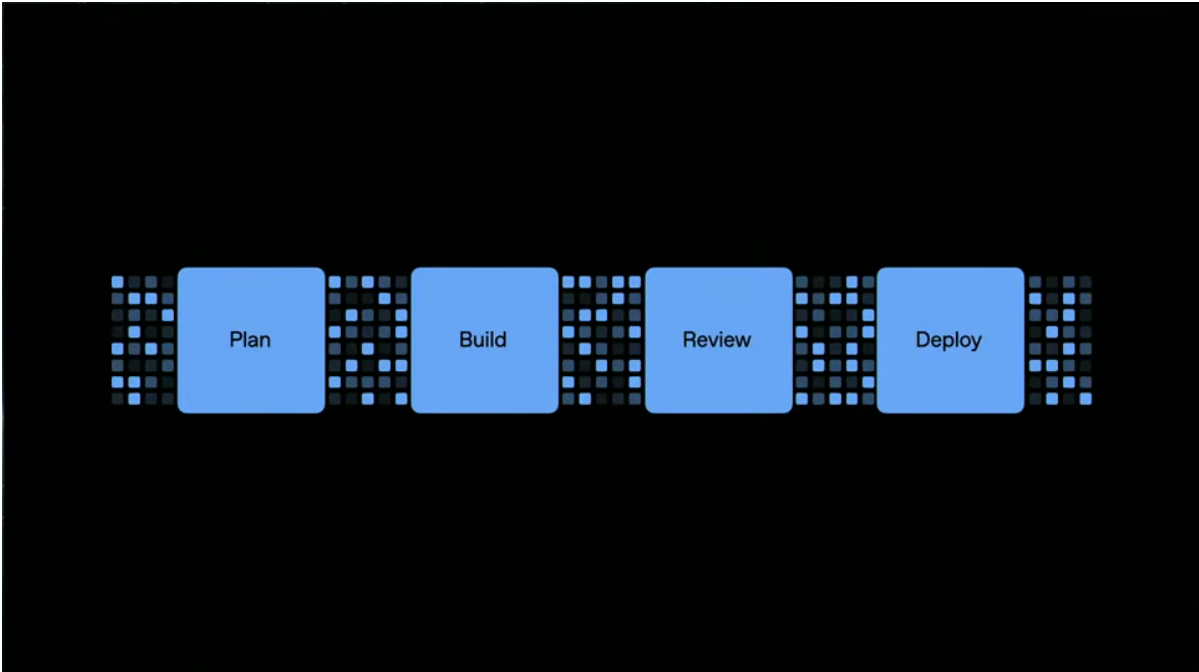


图 3: 演讲画面把代理工作范围放入计划、构建、评审与部署四阶段。原图呈线性链条；工程实践还需要表 3 所列的失败反馈、审批、监控与回退路径。<sup>7</sup>

表 3: 完整交付链中的输入、验证信号与工程师责任

| 阶段 | 进入条件与输入                | 代理可承担的动作与验证信号                          | 工程师最终责任                     |
|----|------------------------|----------------------------------------|-----------------------------|
| 计划 | 用户问题、业务目标、约束、仓库现状、完成标准 | 汇总材料、识别依赖、拆分步骤、提出备选路径；信号是可审查的任务说明与风险清单 | 确认问题值得解决，批准范围、优先级和不可触碰边界    |
| 构建 | 已批准的任务、可复现环境、代码与工具权限   | 修改代码与配置、补充测试、运行构建；信号是差异、构建结果与测试记录      | 约束权限和范围，判断架构与维护成本，处理高风险依赖   |
| 评审 | 可审查差异、验证证据、已知限制        | 解释改动、运行补充检查、回应评审意见；信号是独立检查结果与未决问题      | 进行独立技术和产品判断，防止“生成者自证”成为唯一依据 |
| 部署 | 审批记录、发布策略、运行配置、监控与回退方案 | 准备发布、执行受控步骤、收集运行信号；信号是健康检查、业务指标和告警     | 授权生产变更，保护数据与服务，决定继续、暂停或回退   |

原图中的直线表达作用域扩展，可靠系统需要让反馈逆向流动：评审发现需求偏差时返回计划，构建失败时返回实现，部署观测异常时触发回退并重新界定问题。部署因此不是链条终点；真实环境产生的新证据会进入下一轮计划。

演讲者还指出，更快的执行使产品团队能够制作更多原型，并把更多时间用于理解用户。这个价值来自**扩大有证据的方案搜索**：多个原型提供可比较的行为和反馈，团队据此淘汰弱方案。若原型没有明确问题、用户观察或淘汰标准，数量增加只会积累更多未经判断的产物。

<sup>7</sup>原视频：00:33:21--00:34:08。

## 2.4 闭环成立的隐藏依赖

“把任务交给代理直到完成”听起来像一个操作动作，工程实现中却依赖一份完整的任务契约。可靠的任务委派至少要回答以下问题：

1. **目标与完成标准是什么**：期望行为、允许范围、必须保持的兼容性和可接受误差都需要可检查。
2. **环境是否可复现**：依赖、数据、配置、密钥注入方式和外部服务需要明确，避免把环境故障误判为代码故障。
3. **代理拥有什么权限**：文件、网络、数据库、云资源和部署操作采用最小授权，高影响动作需要单独批准。
4. **验证信号是否可信**：测试、静态检查、评审、运行指标与用户反馈各自覆盖不同风险，单一信号不能包办验收。
5. **失败如何停止和恢复**：限定运行时长、成本、重试次数和改动规模，保留版本点、回滚路径与升级给人工的条件。
6. **谁作出最终决定**：合并、发布、数据迁移和生产权限必须归属于可追责的角色，并保留审计证据。

### 任务委派的最小任务包

一份可委派任务应同时包含：目标、相关上下文、允许使用的工具、环境说明、禁止动作、验证命令、完成标准、停止条件与交付物格式。代理据此持续推进到可评审状态；工程师依据差异、日志和独立验证决定接受、返工或回退。

还有一类风险来自“未知的未知”：测试没有表达的新场景、第三方服务的突发行为、数据分布变化和跨系统耦合都可能落在既有检查之外。工程团队可以通过影子运行、分阶段发布、最小权限、独立评审、监控和快速回退降低影响；这些措施降低风险，无法提供绝对安全保证。

### 高范围任务的工程约束

代理可执行的任务范围受工具适配、操作系统、网络、数据位置、认证授权、运行时限和安全策略共同限制。组织需要自己的环境中完成验证、威胁分析、最小权限设计和部署审批。<sup>8</sup>

## 2.5 本章小结

- 五级能力的分界在于任务单位、反馈能力和责任交接：补全提供候选，闭环提供纠错，委派推进到可评审状态。
- 构建一测试闭环必须包含运行、观察、修正和复验，并由工程师定义可信验证与停止条件。
- 计划、构建、评审与部署需要双向反馈、审批、监控和回退；线性流程图只是作用域概览。
- 任务委派成立的条件是目标、环境、权限、反馈、验收和责任同时明确。

<sup>8</sup>证据范围：演讲以“任何电脑任务”描述能力方向；舞台上的成功路径只证明特定条件下的可行性，未证明代理能在任意环境中可靠完成所有电脑任务。

闭环把更多执行步骤交给智能体。下一章将继续解释工程师如何在目标层沟通与细节层控制之间切换，并对关键决定保持理解、修改和验收能力。

### 3 赋能型协作：从目标沟通到细节控制

#### 本章结论

高效的人机协作需要两种能力同时成立：工程师能够以较低摩擦交代目标，让智能体持续推进；工程师也能够随时进入具体产物，检查、指向、引导和修改。两种能力在同一个任务中反复切换，工程师由此决定何时放手、何时下钻，并对关键决策和最终结果保持理解、控制与责任。

在演讲的使命框架中，产品问题被分成两组：代理能够做什么、应在哪里运行；人应怎样使用代理、产品应呈现什么形态。演讲者随后把产品目标明确界定为尽可能赋能工程师，而非把工程师本身作为自动化对象。<sup>9</sup> 第二组问题由此落到交互设计：界面需要让工程师在委派任务的同时保留理解、介入和验收能力。

前一章说明，任务委派只有在目标、环境、反馈和验收共同可用时才成立。执行闭环扩大之后，新的产品问题随之出现：工程师怎样在保持推进速度的同时理解系统正在做什么，又怎样在关键处重新取得细粒度控制？演讲给出的答案可以压缩为一条协作路径：先用对话建立目标和上下文，再查看智能体形成的可操作产物；随后指向具体对象，请求局部修改或亲手修改；最后依据预先定义的成功标准验收，并决定继续委派、再次介入或停止任务。

#### 3.1 两种可切换的协作状态

**对话式委派 (chat-first collaboration)** 指一种协作顺序：工程师先用自然语言表达目标、约束和意图，接收计划与进展，再按需要进入产物和实现细节。它不要求整个任务停留在聊天窗口。**亲手介入 (hands-on intervention)** 指工程师进入具体对象，执行检查、引导、塑造或直接修改。它是任务中的控制状态，可以在一次委派中出现多次。

演讲画面把这两个状态简化为 Chat 与 Hands on。右侧的工作台照片给出一个直观类比：熟练同事平时可以独立推进；当问题变得关键、含糊或高风险时，协作者会围到同一台设备旁，共同检查部件并调整动作。类比的重点是控制权能够平滑切换。

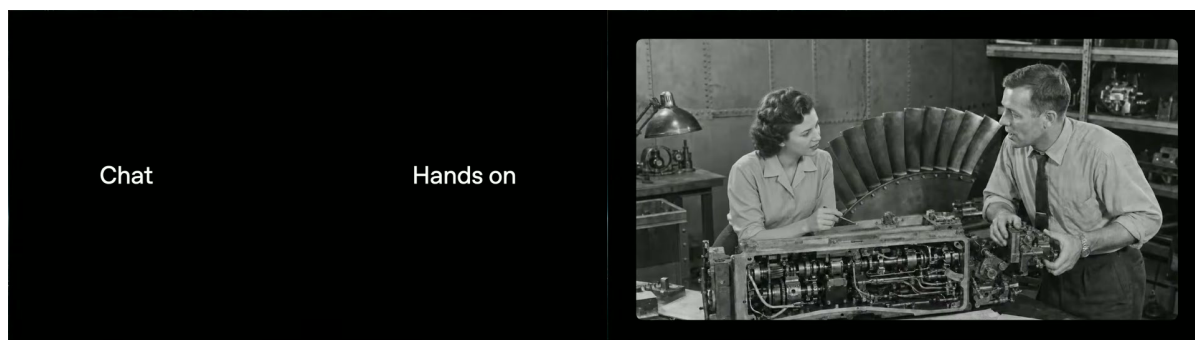


图 4: 画面把 Chat 与 Hands on 并列，并以共同检修装置的工作台场景说明协作状态的切换：日常工作可以委派，关键细节可以共同下钻。该图说明交互模型与类比关系，不证明特定设备或人物身份。<sup>10</sup>

<sup>9</sup> 原视频：00:35:10–00:35:47。证据范围：这是演讲者对产品方向的陈述，不构成就业变化或普遍自动化程度的预测。



两种状态解决的问题不同，组合后才形成可持续协作。下表从五个维度界定两者的职责。

| 比较维度  | 对话式委派                                 | 亲手介入                                  |
|-------|---------------------------------------|---------------------------------------|
| 主要目的  | 交代目标、澄清意图、补充约束、接收计划和进展                | 检查具体产物、定位局部问题、改变方向或直接完成修改             |
| 信息粒度  | 任务、用户需求、成功标准、优先级与状态摘要                 | 界面元素、文件、代码片段、运行结果和具体失败信号              |
| 工程师动作 | 提问、说明背景、批准计划、追加约束、要求继续或暂停             | 查看、选中、指向、批注、局部改动、接管操作和复验              |
| 适用时机  | 目标尚需澄清、任务可持续推进、希望低成本了解全局状态时           | 决策影响较大、结果偏离意图、局部细节决定质量或必须人工签字时        |
| 主要风险  | 目标含糊会被持续放大；状态摘要可能遗漏关键细节；过久不检查会形成不透明委派 | 过早进入细节会打断连续推进；频繁手工改动可能造成上下文和当前产物状态不同步 |

表 4: 两种协作状态的职责比较。它们属于同一任务中的可切换控制状态。

一次实际任务通常沿着“委派—观察—介入—再委派”推进。例如，工程师先说明“改善世界杯预测应用的使用体验”，智能体提出改进计划并开始构建；工程师看到结果后，可能只对比分卡片、提醒方式或排名显示提出局部要求；局部修改通过验证后，剩余工作又可以交回智能体。切换点来自任务风险和信息需要，不来自固定的界面步骤数。

### 3.2 理解、控制与责任来自哪些操作

演讲把产品目标表述为让工程师保留**工作掌控感 (mastery)**。这里的“掌控”同时包含理解、控制、判断和责任，其范围大于操作熟练度。工作掌控感也不等于要求工程师逐行监视全部执行过程；它依赖一组能够在关键时刻使用的产品动作。

1. **看见当前对象。**工程师能够区分任务目标、智能体计划、正在运行的产物和待处理的修改，知道讨论对象处于什么状态。
2. **追踪关键决定。**界面应让工程师理解智能体选择了哪条路径、哪些约束已经被采用、哪些问题仍未解决。<sup>11</sup>
3. **指向具体位置。**工程师可以把反馈绑定到某个界面区域或实现对象，降低“把这里改好一些”一类含糊指令造成的误差。
4. **改变局部结果。**工程师可以请求智能体只改指定部分，也可以亲手完成修改。局部控制有助于保留已经正确的部分，并缩小重新验证的范围。
5. **决定是否通过。**可运行界面、测试结果和差异视图都是验证信号；最终验收仍需回到任务的成功标准、风险等级和责任归属。

<sup>10</sup>原视频：00:35:58–00:37:12。

<sup>11</sup>证据范围：演讲展示了产品方向，没有给出完整审计日志格式。

这些动作构成一条控制链：可见性提供判断材料，可指向性把判断落到对象，可修改性提供纠偏能力，验收权决定结果能否进入下一阶段。任何一环缺失都会削弱协作。例如，工程师能看到漂亮的运行界面，却无法追踪数据来源和测试范围，当前信息仍不足以支持可靠验收；工程师能直接改代码，却无法把改动同步给正在执行的智能体，后续任务可能继续基于过期状态。

### 工程落地控制清单

工程团队应把以下控制纳入任务验收基线：记录任务状态与改动差异；保留工具调用、测试证据、权限使用和中止原因；提供版本化、回滚与重新运行能力。<sup>12</sup>

工作掌控感由此可以被理解为“随时能够获得足够信息，并采取与责任相匹配的动作”。低风险探索可以保持较长时间的对话式委派；生产部署、数据迁移、权限变更等高风险步骤需要更早、更细的人工介入。任务风险决定控制密度，界面负责让这种密度可调。

## 3.3 Codex App 的下钻路径

Codex App 把上述控制链组织成“对话先行、需要时下钻”的产品路径。演讲中的世界杯预测应用展示了同一工作区里的两个连续状态：首先是围绕项目目标形成的改进计划，随后是可运行、可检查、可继续修改的 Web 产物。

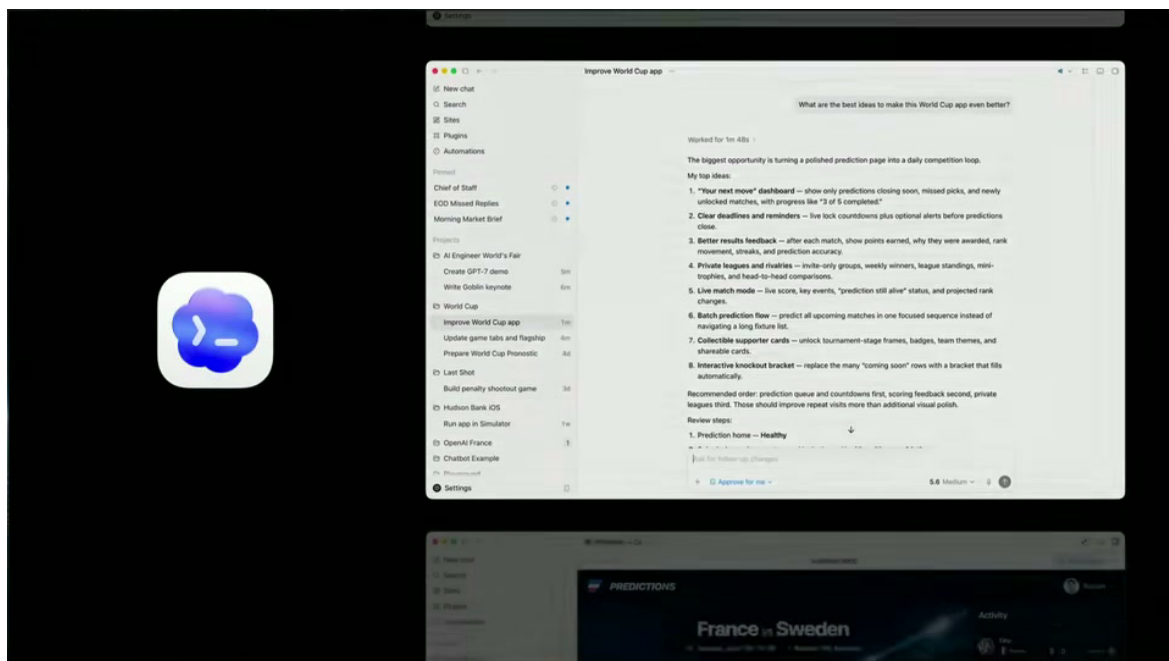


图 5: Codex App 的对话与规划状态：画面可见项目导航、会话内容和一组针对世界杯应用的改进建议，用于说明目标如何被整理为后续可执行工作。该页证明界面中出现了计划，不证明所有建议均已实现或验证。<sup>13</sup>

这条下钻路径包含六个连续关口。

<sup>12</sup>证据范围：演讲现场主要展示检查、引导、塑造与直接修改；上述日志、恢复和版本控制项是工程落地要求，未被作为 Codex App 的现场功能清单展示。

<sup>13</sup>原视频：00:37:12–00:37:31。



| 关口    | 界面中的对象               | 工程师需要确认的内容                | 进入下一步的信号              |
|-------|----------------------|---------------------------|-----------------------|
| 目标对话  | 需求说明、背景、约束与问题澄清      | 目标是否具体；禁止事项、优先级和成功标准是否完整  | 智能体能够复述任务边界，并形成可检查的计划 |
| 可操作产物 | 计划、预览、运行界面、文件或改动结果   | 产物是否对应目标；当前完成范围与未决问题是否清楚  | 结果能够被查看、运行或比较，且对象身份明确 |
| 定点指向  | 被选中的界面区域、文件或局部实现     | 反馈是否绑定到唯一对象；保留部分与修改部分是否分开 | 智能体和工程师对“改哪里”形成同一理解   |
| 局部修改  | 智能体改动或工程师亲手改动        | 修改是否越过边界；相关状态、依赖与测试是否同步更新 | 新版本可重新运行，并能与修改前结果比较   |
| 验证与验收 | 运行结果、测试、差异、评审意见和风险清单 | 成功标准是否满足；剩余失败能否接受；谁承担发布责任 | 明确作出接受、返工、中止或回退决定     |
| 继续委派  | 新目标、遗留问题与下一轮约束       | 已验收结果是否成为稳定基线；新增目标是否引入新权限 | 下一轮任务拥有清楚的输入、边界和停止条件  |

表 5: Codex App 协作路径拆解：从对话、产物、指向和修改进入验证与验收。

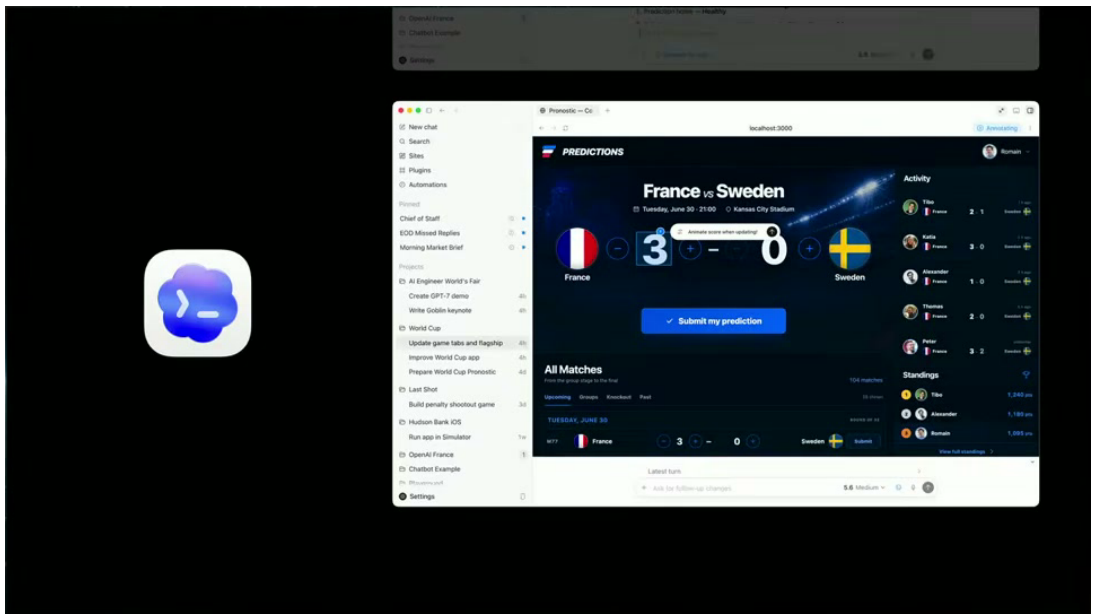


图 6: Codex App 中可运行的世界杯预测应用：左侧保留项目与任务上下文，右侧呈现可检查的 Web 结果和修改入口，说明对话如何下钻到具体产物。比分、活动记录与排名均为演示界面内容。

<sup>14</sup>原视频：00:37:23–00:37:43。画面只证明演示应用呈现相应界面与数据；任务成功率、测试覆盖、生产可靠性和完整验收机制均未被展示。

演示呈现了“从会话进入产物、指向局部、请求修改或亲手修改”的操作路径。可运行预览能够回答“结果是否可见、交互是否接近意图”；测试、数据正确性、权限审查、性能和发布审批继续回答“结果是否足以交付”。两类证据合并后，才形成从对话到验收的闭环。

### 3.4 CLI、IDE 与 Codex App 的入口选择

演讲者用命令行界面（CLI）、集成开发环境（IDE）和 Codex App 说明三种不同的产品起点。他的判断是：CLI 留给持续可视协作的界面空间有限；IDE 往往从代码对象开始；Codex App 希望先建立对话式委派，再在需要时进入代码和其他计算机产物。这段比较解释了 Codex App 的入口设计。<sup>15</sup>

CLI 可以提供脚本化、可组合和低层控制，IDE 可以提供代码导航、调试和局部编辑；Codex App 的论点集中在入口顺序和状态切换：用户可以从目标开始，随后进入产物。具体团队仍可将 CLI、IDE、扩展和 App 组合进同一个工作流，并依据自动化程度、可观察性、代码导航、工具链、权限、审计要求和使用者习惯选择入口。

演讲者还提到，早期终端、Vim、Emacs、CLI 和 IDE 用户曾对 Codex App 表示质疑，其中一些人后来开始使用它。<sup>16</sup> 这一观察说明，对话先行的产品形态存在接受门槛，可操作产物与下钻控制能够帮助使用者判断它是否适合自己的工作流。

### 3.5 本章小结

- 对话式委派负责建立目标、上下文和持续沟通；亲手介入负责检查、定点纠偏、直接修改和作出关键判断，两者在同一任务中循环切换。
- 工作掌控感来自可见、可追踪、可指向、可修改和可验收的操作权，其含义包含控制与责任。
- Codex App 展示了“目标对话—可操作产物—定点指向—局部修改”的下钻路径；可靠交付还需工程师补足验证信号和验收标准。
- CLI、IDE 与 Codex App 代表不同的入口顺序。团队应依据任务、工具链和治理条件组合协作界面。

协作界面解决了工程师如何放手与重新介入的问题。下一章将继续向下展开，说明模型、Responses API、任务执行层、App Server、应用与插件怎样共同支撑这种协作。

## 4 开放代理栈：从模型到插件的六层结构

### 本章结论

开放代理栈通过六层职责把模型能力转化为可运行、可控制、可复用和可扩展的产品。Models 提供基础能力，Responses API 暴露调用与共用原语，Codex Harness 承载智能体运行，App Server 连接客户端，App 组织用户协作，Plugins 增加专业能力。各层的开放方式并不相同；开发者需要同时看见可使用的接口和仍由服务、权限或内部系统控制的边界。

<sup>15</sup>证据范围：CLI、IDE 与 Codex App 的比较属于演讲者的产品设计判断；演讲没有系统比较三类入口，也没有提供受控实验、用户研究样本、成功率或完成时间来证明某一入口在所有任务、仓库和团队中更高效，因此不能作为通用工具优劣排名。

<sup>16</sup>证据范围：这是演讲者对产品采用过程的概括，没有总体用户偏好调查；它只支持“部分怀疑者重新评估了该产品形态”这一有限结论。

上一章从用户视角说明了对话、产物、指向、修改和验收。该交互链的背后需要多种职责配合：模型要能推理和调用工具，运行系统要维持多步任务，客户端要接收状态并提供控制入口，插件还要在受控条件下增加新工具。演讲用六层栈把这些职责组织起来，并提出“内部产品与开发者使用同源组件”的开放生态主张。

## 4.1 六层结构与职责边界

演讲画面自下而上列出六层。底部三层是 Models、Responses API 与 Codex Harness；其上是 App Server、App 与 Plugins。这种排列表达能力如何逐层被封装：底层回答“系统具有什么能力”，中层回答“能力怎样持续运行并被客户端控制”，上层回答“用户怎样协作以及第三方怎样增加新能力”。它是一张职责模型，不要求部署时恰好出现六个独立网络服务。

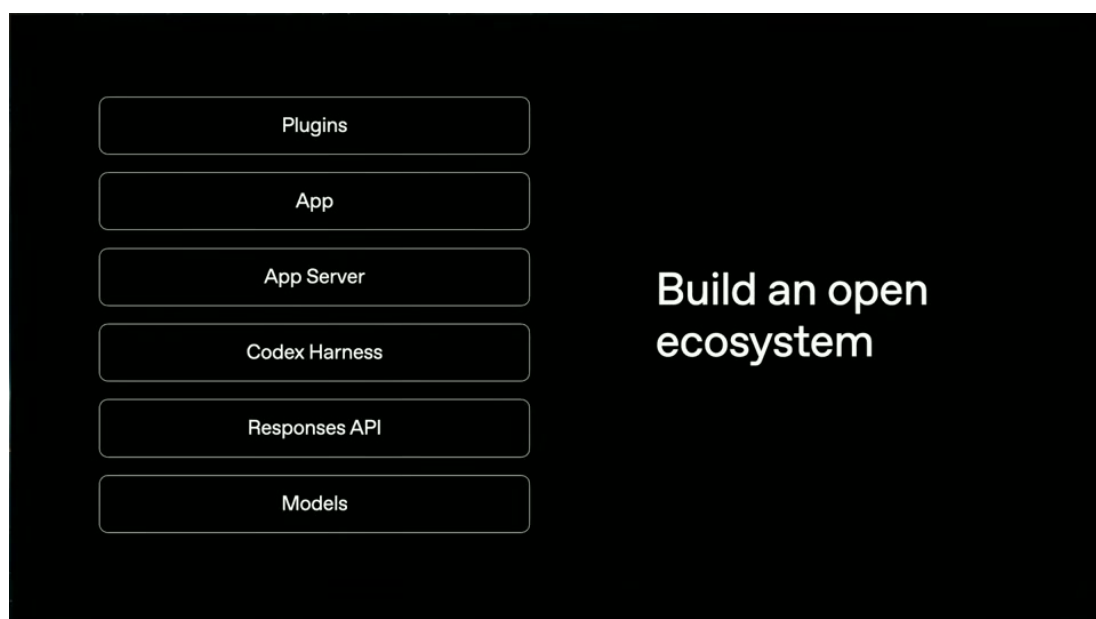


图 7: 演讲给出的开放生态六层总览：Models、Responses API、Codex Harness、App Server、App 与 Plugins 自下而上形成能力、运行、控制和体验层。右侧“Build an open ecosystem”是演讲的产品主张。<sup>17</sup>

下表把每一层的主要职责、演讲所述开放方式和最容易混淆的边界放在一起。这里的“开放”包含托管访问、公开接口、开放源代码和插件接入等不同形态，不能用一个词覆盖全部层级。

<sup>17</sup>原视频：00:38:58–00:39:42。

| 层级            | 主要职责                                                      | 演讲所述开放方式                    | 职责边界                                         |
|---------------|-----------------------------------------------------------|-----------------------------|----------------------------------------------|
| Models        | 提供推理、生成、工具调用决策等基础能力                                       | 开发者经同一类模型接口使用托管能力           | 模型调用本身不维持完整智能体循环 (agent loop); 演讲没有说明模型权重开放  |
| Responses API | 暴露模型能力和 Codex 所需的共用原语, 例如长任务相关的压缩能力                       | 以开发接口供 Codex App 和外部产品调用    | API 规定能力入口, 不负责完整用户界面, 也不承载任务执行循环            |
| Codex Harness | 承载智能体循环、工具协议、环境访问、状态管理和执行控制                               | 演讲称其开放源代码, 可检查、分叉和改造        | 智能体运行框架 (agent harness) 不等于底层模型, 也不等于面向用户的应用 |
| App Server    | 向 VS Code 扩展、Codex App 等客户端提供统一的 Harness 控制路径, 并传递会话与任务状态 | 演讲称自家产品采用的路径已经开放源代码         | 该专名表示代理应用控制层, 不能按普通 Web 服务器理解; 具体部署与认证细节未披露  |
| App           | 组织对话、产物、进度、局部控制和用户 workflow                               | Codex App 展示内部产品如何消费同一套下层能力 | 应用负责体验和控制入口, 底层权限、模型与运行可靠性仍由相邻层共同决定          |
| Plugins       | 加入浏览器操作、计算机操作、知识和面向角色的流程                                  | 通过面向内部与外部开发者的相同接入边界扩展       | 插件接入不自动获得无限权限; 认证、授权、隔离和数据范围仍需系统约束           |

表 6: 开放代理栈的六层职责表。表中开放方式依据演讲陈述, 具体版本和可用范围仍受产品条件约束。

从调用方向看, App 和 Plugins 把用户目标转换为客户端动作, App Server 把会话控制送到 Codex Harness, Harness 再借助 Responses API、模型与工具推进任务; 运行状态和产物沿相反方向回到用户界面。实际实现还可能包含队列、数据库、权限服务、环境访问控制和观测系统。演讲没有把这些基础设施画入六层图; 这些系统仍是落地依赖, 其实现不能由六层图直接推断。

## 4.2 多步执行怎样持续运行

**智能体循环**指围绕目标反复执行行动、工具调用、结果观察和策略修正的运行过程。一次模型响应可以提出方案或生成内容; 持续任务还需要保存状态、调用外部工具、读取真实反馈并判断下一步。一个便于实现和审查的循环可以拆成以下步骤:

1. **读取目标与当前状态。**系统取得任务说明、项目指令、现有产物、权限和停止条件。
2. **选择下一动作。**模型根据当前状态决定查询信息、编辑文件、运行命令、请求澄清或结束任务。
3. **执行工具调用。**运行层把结构化动作交给文件系统、终端、浏览器或其他工具, 并限制可访问范围。
4. **观察真实结果。**构建错误、测试结果、页面状态和工具返回值进入下一轮上下文。

5. **修正或收束**。系统根据反馈调整计划，直到满足停止条件、需要人工批准、发生不可恢复错误或达到资源限制。

**智能体运行框架**是承载这条循环的工程外壳。它把模型选择与任务运行职责分开，管理工具协议、环境访问、状态和执行控制。演讲中的 **Codex Harness** 是这一一般工程角色的具体实现。

演讲提供了三项关键边界。第一，Codex Harness 默认使用 OpenAI 模型，模型提供方没有被硬编码；外部团队可以替换模型，同时复用智能体循环。第二，**AGENTS.md** 被选择为多种代理可以共同采用的项目指令文件约定，意图是减少产品专属指令格式造成的生态分裂。第三，同一 Harness 进入模型后训练流程，使模型在公开可检查的环境中学习工具调用和环境导航。<sup>18</sup>

智能体循环还需要明确的停止规则。任务完成、验证通过、需要高风险权限、连续失败、预算耗尽和人工中止都可以成为停止信号。演讲强调持续完成复杂目标，但没有定义重试上限、权限升级或事故处理流程；这些规则应由具体产品和组织治理补足。

### 4.3 长时任务怎样保留关键状态

长时任务会不断积累对话、工具返回、文件内容、错误日志和中间决策。全部历史持续进入模型会消耗有限的上下文空间，也会提高处理成本。**上下文压缩 (context compaction)** 把过长历史整理成较短表示，使任务能够释放空间并继续推进。它可以保留目标、关键约束、当前计划、已验证事实和未决问题，同时降低重复日志与低价值细节的占用。

演讲把上下文压缩作为同源构建的具体例子：Codex 的长任务需要该能力，团队随后把它加入 Responses API，让外部智能体也能使用相同原语。这个例子说明产品需求可以下沉为开发接口。演讲没有披露压缩算法、容量上限、触发阈值、恢复机制或保真率。

#### 上下文压缩与简单截断的区别

简单截断直接丢弃一段旧内容，系统可能同时失去目标、约束和决策理由。上下文压缩尝试把重要状态重写为较短表示，仍需决定哪些信息具有长期价值。在工程实现中，原始日志、结构化状态和压缩摘要通常需要分层保存，以便复核摘要和恢复关键证据。

上下文压缩带来三类风险。其一，细节丢失会让智能体重复已经失败的路径；其二，摘要偏差会把暂定判断写成确定事实；其三，安全约束和验收条件可能在多轮压缩中被弱化。可靠系统应把不可丢失的指令、权限边界和验收条件放入稳定状态，记录压缩前后的来源关系，并允许人工检查关键摘要。<sup>19</sup>

### 4.4 App Server 如何连接客户端

Codex Harness 负责运行任务，上层客户端还需要统一方式来创建会话、发送控制动作、接收进展并展示结果。演讲把这一职责放在 **App Server**<sup>20</sup>。该层源于 OpenAI 构建 VS Code 扩展与 Codex App 的实际需要；演讲者把它描述为“自家产品实际采用的路径”，并称该路径已经开放源代码。

<sup>18</sup> “模型未硬编码” 仅表明智能体运行框架具有替换模型的设计空间，任意替代模型的工具使用效果仍需单独验证。“Harness 进入后训练流程” 只证明训练环境与产品智能体运行框架存在连接；演讲未公开完整训练配方、训练数据、评测方法或内部部署配置，模型权重也不在 Harness 的开放范围内。

<sup>19</sup> 演讲明确陈述上下文压缩已进入 Responses API；稳定状态、来源关系与人工检查是据此提出的工程要求，演讲未公开这些机制的具体实现。

<sup>20</sup> App Server 是演讲六层栈中的正式组件名，表示代理应用的统一控制路径，语义范围超出普通静态 Web 服务。演讲未完整说明会话持久化、认证授权、并发隔离、故障恢复和部署拓扑，这些实现不能从组件名称直接推断。



可以把 App Server 理解为客户端与智能体运行框架之间的控制边界。客户端提交目标、继续、暂停、批准或局部修改等动作；App Server 把动作转换为 Harness 可处理的会话和任务控制；Harness 返回状态、工具结果和产物引用；客户端再把这些信息组织成对话、差异或预览。这个职责解释了为何多个客户端可以共享底层任务能力，同时保留各自的交互形态。

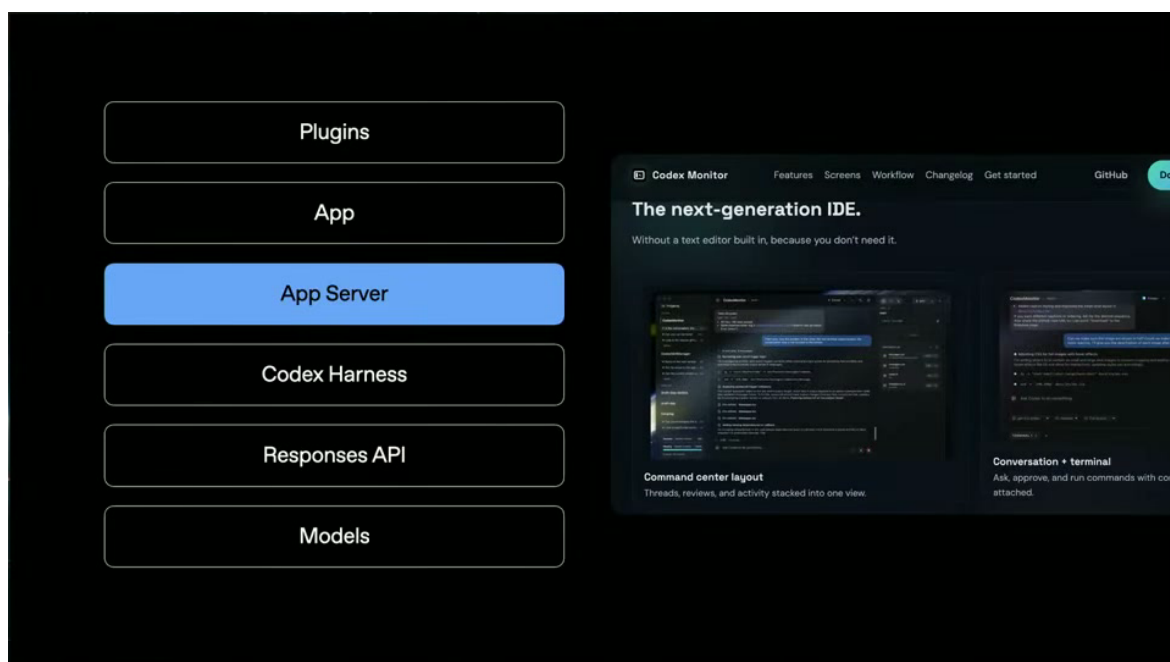


图 8: 画面高亮 App Server，并以 Codex Monitor 页面展示上层应用案例。该图说明 App Server 位于 Harness 与客户端之间，案例页面证明演讲把 Codex Monitor 与该层联系起来；页面宣传语不构成产品质量证据。<sup>21</sup>

Codex Monitor 提供了一个外部复用案例。演讲称，第三方开发者曾在 Codex App 发布前基于 App Server 构建原生应用；该开发者后来加入团队并参与 Codex for iOS。<sup>22</sup> 该案例能够证明 App Server 被外部产品采用的演讲叙述，不能推出所有客户端功能、认证方式和订阅条件都已经标准化。

#### 4.5 开放组件怎样形成生态反馈

开放组件为外部团队提供**参考实现 (reference implementation)**：它是一套可运行、可检查、可复用和可分叉的实现样板，帮助开发者理解预期行为并选择保留或替换的部分。参考实现提供共同起点，不构成唯一正确实现或强制产品规范。

演讲以 OpenCode 说明这种用法。外部团队可以阅读 Codex Harness 的实现，复用合适部分，改变其余设计；演讲还提到，通过代码了解 ChatGPT 登录路径可以减少对产品行为的逆向工程。图中 OpenCode 页面与 Harness 高亮同时出现，直接支持“第三方工具参考开放组件”的案例归属。页面写有“The open source AI coding agent”，该英文是项目页面自身表述，不能证明其完整能力。

<sup>21</sup> 原视频：00:41:00–00:41:49。

<sup>22</sup> 开发者姓名与账号无法由字幕和画面稳定核验，因此省略。

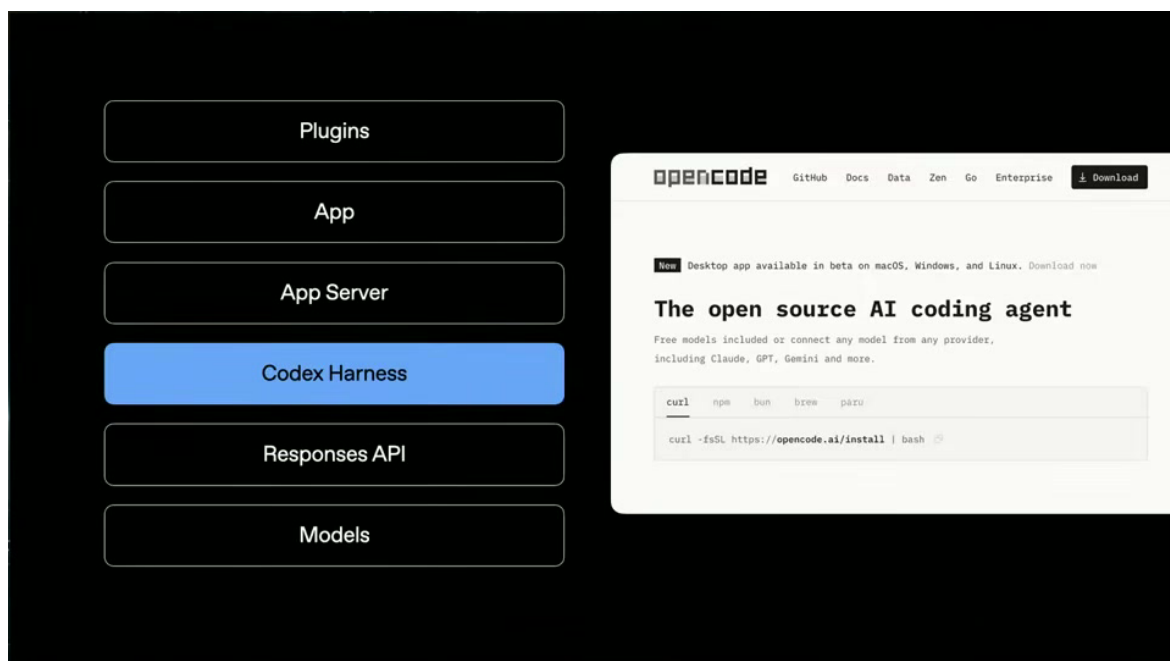


图 9: 画面高亮 Codex Harness, 并并列展示 OpenCode 项目页面, 用于说明外部团队能够检查智能体运行框架的实现、选择复用部分并调整其他设计。该案例说明开放复用路径, 不代表 OpenCode 与 Codex 产品完全相同。<sup>23</sup>

应用层通过**扩展点 (extension point)** 加入新能力。扩展点是平台预留的稳定接入边界, 插件可以借此增加工具、知识或专业流程。演讲称 browser use、computer use 和面向数据科学、设计等角色的插件使用对外开放的相同扩展点, 并把这些插件作为开放生态的一部分。

插件的可扩展性需要和权限治理共同设计。浏览器操作可能接触登录状态和网页数据, 计算机操作可能影响本地文件与其他应用, 面向角色的插件可能引入组织知识或外部服务。扩展点只能定义“怎样接入”; 认证、授权、数据范围、隔离、审计和撤销继续由平台与组织控制。开放接入也不保证某个插件在所有版本、地区和订阅层级中可用。

<sup>23</sup>原视频: 00:40:32–00:40:57。

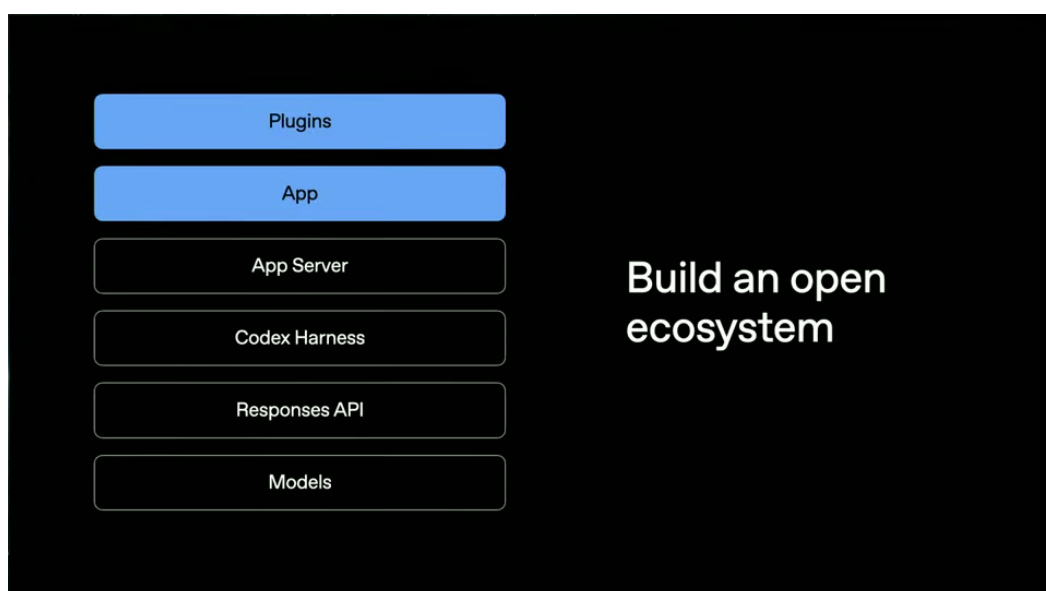


图 10: App 与 Plugins 两层同时高亮，右侧标注“Build an open ecosystem”。同期口述称扩展点可增加浏览器操作、计算机操作或面向角色的能力。<sup>24</sup>

生态反馈由此形成回路：内部产品把共用能力下沉到 API、Harness、App Server 或扩展点；外部团队检查、分叉和组合这些能力；真实使用触及能力边界并产生反馈；产品团队再决定哪些需求进入公共层。演讲列举 OpenCode、Pi、Droid、OpenClaw、Xcode 与 JetBrains 等入口，用于说明多入口方向。清晰可证的结论是演讲展示了这些生态示例，当前可用性和订阅条件仍需另行核实。

#### 4.6 同源构建主张的可信边界

演讲的最高层主张是：OpenAI 没有为自身维护一套完整产品栈，再为开发者提供另一套简化栈；团队在各层使用交付给开发者的组件或接口。当前证据可以支持四项具体含义：Codex App 经相同 Models 与 Responses API 构建；Codex Harness 可供外部检查和改造；App Server 是 VS Code 扩展与 Codex App 的实际路径；内部插件与外部插件使用相同扩展点。

“同源”限定于产品接口和演讲明确列出的组件关系；取得同源接口或代码后，运行环境、资源、服务等级和治理策略仍按各自条件配置。

<sup>24</sup>原视频：00:41:48–00:42:29。

| 开放形态  | 演讲中的对象                                 | 开发者能够做什么                      | 仍然存在的边界                      |
|-------|----------------------------------------|-------------------------------|------------------------------|
| 开放源代码 | Codex Harness、App Server，以及演讲提到的面向角色插件 | 检查实现、分叉代码、按需改造并组合进外部产品        | 不包含模型权重、全部产品代码、内部基础设施或服务运行保证 |
| 开放接口  | Responses API，以及模型和共用原语的调用路径           | 通过稳定请求和响应边界调用托管能力，把相同原语接入自有系统 | 受版本、配额、价格、地区、认证和服务可用性约束      |
| 可扩展点  | App 与 Plugins 之间的工具和流程接入边界             | 增加浏览器操作、计算机操作、知识或角色工作流        | 不授予无限权限；数据访问、授权、隔离、审核和撤销仍受控制 |
| 托管能力  | Models 及通过 Responses API 提供的服务能力       | 使用演讲所称与 Codex App 同源的模型能力和原语  | 实现细节、权重、训练数据、容量策略和完整部署配置未被公开 |

表 7: 四种开放形态的边界。开放源代码、开放接口、扩展点和托管能力提供不同层级的参与方式。  
25

这一区分给架构设计带来一个直接方法：阅读开放栈时，需要逐层提出四个问题——可检查的对象是什么、可调用的接口是什么、需要谁提供托管服务、权限最终由谁控制。只有四个问题都有答案，团队才能判断一项能力能否被替换、部署、审计和长期维护。

4.7 本章小结

- 六层栈把基础能力、开发接口、智能体运行、客户端控制、用户体验和插件扩展分开；它描述职责关系，不规定六个独立网络服务。
- 智能体循环依赖工具调用、真实反馈和停止规则，智能体运行框架负责承载这条循环；模型与智能体运行框架可以在职责上解耦。
- 上下文压缩支持长时任务继续推进，同时带来细节丢失、摘要偏差和约束弱化风险；稳定状态与来源记录仍需工程实现补足。
- OpenCode、Codex Monitor 和插件案例说明参考实现、应用控制路径与扩展点如何被外部复用；案例不能外推为普遍可用性保证。
- 同源构建的可信范围限于演讲明确列出的模型接口、Responses API、Codex Harness、App Server 和扩展点，内部基础设施与治理边界仍未披露。

开放代理栈解释了协作能力怎样进入产品。下一章将从成本、速度和任务运行位置继续判断这些能力能否在真实工程中形成可持续价值。

<sup>25</sup> “同源构建”是演讲者对模型接口、Responses API、Codex Harness、App Server 和扩展点的产品陈述。演讲未证明内部基础设施、权限、安全策略、训练数据和完整部署实现与外部环境完全一致；各生态入口的版本、地区和订阅层级可用性仍需按实际产品条件核验。

## 5 智能体价值：成本、速度与任务运行位置

本章结论

智能体的价值应按完整任务核算：有用结果是否通过验收、总成本是否可承受、等待时间是否满足工作节奏、执行环境是否符合数据与权限约束。模型消耗更多 token 只说明计算活动增加，无法单独证明工程价值增加。

### 5.1 怎样评价完整任务的价值

价值最大化（value maxxng，现场用语）是演讲者对“怎样从越来越有用的智能体中取得实际收益”的概括。这个说法没有正式公式、统一单位或公开的综合测量方法。可执行的理解方式是把它拆成成本、速度和任务运行位置三个决策维度，并把验收质量作为贯穿三者的硬约束。

三个维度回答不同问题。成本决定团队能够持续承担多大的任务范围；速度决定一次工作循环中可以获得多少反馈和候选方案；运行位置决定任务能否取得所需数据、工具、硬件与持续运行能力。任何一项都需要回到完整任务观察，单一基准、单价或生成吞吐量无法替代端到端验收。

表 8: 价值最大化的三维决策框架

| 维度   | 核心问题               | 应观察的任务信号                       | 常见误读                 |
|------|--------------------|--------------------------------|----------------------|
| 成本   | 完成并验收一个任务需要多少资源    | 模型调用、工具与计算、重试、并行候选、人工评审、失败返工   | 只比较每 token 单价或一次调用价格 |
| 速度   | 从提出目标到获得可验收结果要等待多久 | 首次反馈、工具等待、构建与测试、候选比较、人工决策、部署准备 | 把生成吞吐量当作完整任务速度       |
| 运行位置 | 哪个环境能在约束内稳定完成任务    | 数据位置、权限、硬件、网络、持续运行、隔离、回退与可用性   | 把本地或云端视为永久且统一的选择     |

这张表的使用顺序是先写清验收标准，再估计三维代价。若验收目标含糊，低成本和高速度只会更快地产生无法判断的输出；若运行位置违反数据或权限边界，基准成绩再高也无法进入真实交付。

### 5.2 成本与高能力模型的现场比较

演讲者使用“前沿智能水平（frontier intelligence）”描述会议时点的高能力模型层级。该名称属于产品化用语，没有固定公开阈值。TerminalBench 2.1 的现场柱状图提供了一个工程任务基准横截面，可以比较图中配置在该评测上的成绩；它没有覆盖特定代码库、组织权限、长时任务、人工返工或部署风险。



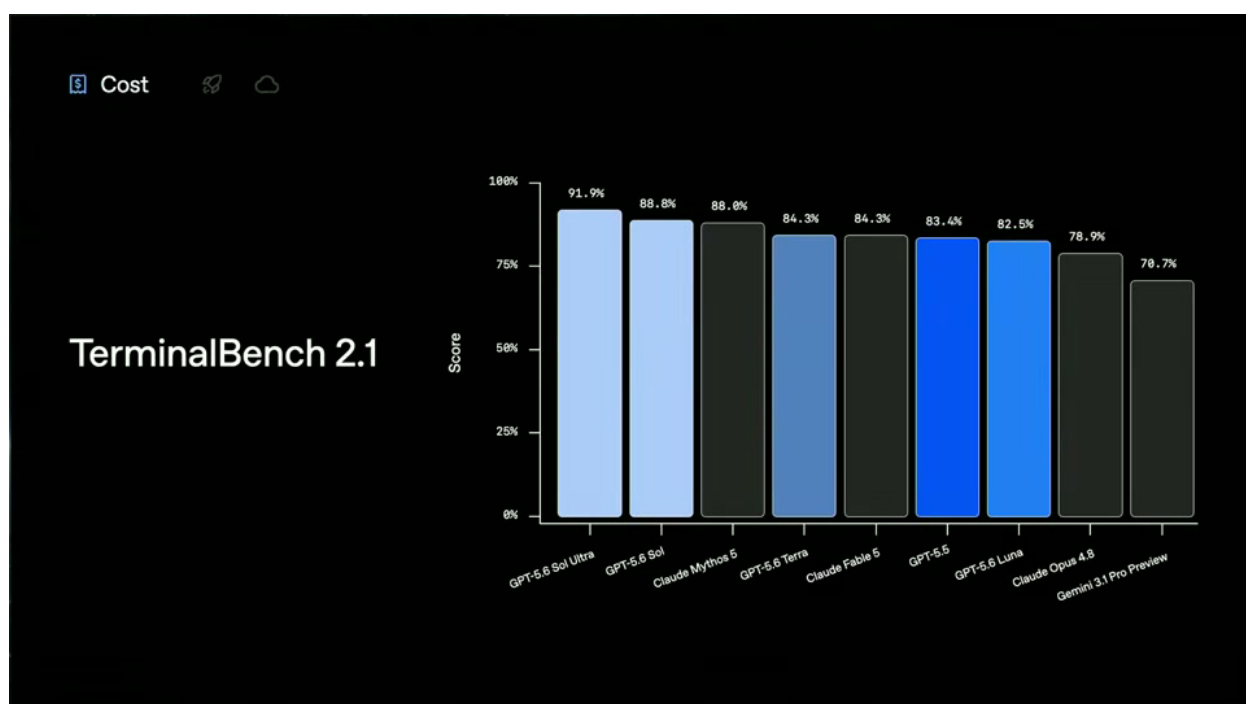


图 11: 成本维度的现场证据: TerminalBench 2.1 柱状图展示多种模型与配置的基准得分, 用于说明同一能力讨论中仍需考虑成本效率; 该图属于会议现场快照。<sup>26</sup>

图中可清晰辨认的 OpenAI 型号与得分包括 GPT-5.6 Sol Ultra 的 91.9%、GPT-5.6 Sol 的 88.8%、GPT-5.6 Terra 的 84.3%、GPT-5.5 的 83.4% 和 GPT-5.6 Luna 的 82.5%。演讲者同时称 Terra 提供接近 GPT-5.5 的智能水平, 成本约为一半; 其对 Luna 的现场价格陈述为每百万输入 token 1 美元、每百万输出 token 6 美元, 并以该评测中的相对位置说明成本效率。

完整任务成本还包括模型单价之外的支出。长上下文会增加输入量; 失败重试会重复消耗; 五到六条候选路径会扩大并行计算; 工具调用、编译、测试和远程环境会占用额外资源; 工程师还要承担评审、合并、回退与事故处置时间。因此, 成本效率的正确问题是“一个通过验收的结果总共消耗了什么”, 并应同时记录失败样本和人工介入。

### 5.3 速度如何扩大方案搜索

速度首先缩短单路反馈, 其更深的工程意义是扩大同一时间预算内的方案搜索。演讲者先以 GPT-5.3 Codex Spark 说明速度可能改变代理交互, 再以 GPT-5.6 Sol on Cerebras 的现场示例把讨论落到生成吞吐量。<sup>27</sup> 若生成足够快, 智能体可以并行尝试五到六条路径, 再由比较机制或工程师挑选更合适的结果。这里的“五到六条”属于现场举例, 并行度、比较方法、质量稳定性与资源成本均未被规定。

<sup>26</sup>原视频: 00:44:16–00:45:13。图中型号、得分、价格与相对比较均来自会议画面或演讲者口述; 完整评测设置、价格条件、缓存计价、批量折扣、地区差异、发布日期和后续变更未获独立复核。这些信息只说明演讲现场的产品论证, 不构成当前规格、采购报价或普遍性能保证。

<sup>27</sup>GPT-5.3 Codex Spark 的来源区间: 原视频 00:45:13–00:45:21。型号名称与产品状态来自会议现场陈述, 后续命名、发布状态和可用范围未获独立复核; 该例只支持“速度变化可能改变交互方式”, 不代表当前型号状态或当前产品规格。



图 12: 速度维度的现场示例：画面显示 GPT-5.6 Sol on Cerebras 与 750 tokens/s，演讲据此讨论更快反馈和并行方案搜索；该图属于会议现场快照。<sup>28</sup>

#### 生成吞吐量衡量什么

每秒 token 数衡量模型持续生成文本或代码的速率。完整任务还要经历首个结果等待、工具调用、网络传输、依赖安装、编译、测试、候选比较和人工验收。演讲者用“约十秒写出一个相当可观的 PR”建立速度直觉；该说法缺少 PR 大小、测试范围和验收标准，只适合作为现场类比。

速度转化为价值需要四个环节衔接。第一，任务能够被拆成相互区分的候选路径。第二，每条路径在受控环境中运行，并保留日志、修改集和验证结果。第三，比较机制使用同一验收标准筛选候选。第四，工程师对最终方案的风险、可维护性和部署影响作出判断。缺少比较与验证时，更高吞吐量只会加速产生更多未经证实的候选。

### 5.4 多任务怎样避免相互干扰

当任务从几秒钟的代码建议扩大到长时构建、测试和多路径探索时，共享一个无边界工作区会产生文件冲突、依赖污染、端口争用和凭据扩散。隔离执行环境（isolated environment）为每个任务提供独立工作区、容器、虚拟机或远程执行单元，使任务能够并行推进，并把访问范围、资源配额和停止条件绑定到单个任务。

在工程实现中，隔离执行环境至少需要下列控制：

- **文件与依赖边界：**每个任务拥有独立修改集和依赖状态，合并前可比较差异并丢弃失败分支。
- **权限边界：**工具、网络、凭据和敏感数据按任务最小授权，临时凭据应能撤销和过期。

<sup>28</sup> 原视频：00:45:13–00:46:18。750 tokens/s 是画面针对 GPT-5.6 Sol on Cerebras 的生成吞吐量陈述，属于会议现场快照；该数字不含工具调用、编译、测试、部署和人工评审耗时，也未披露并行候选的总成本，端到端任务速度仍需另行测量。

- **资源边界：**限制运行时长、并发数、计算和存储，防止失败循环无限消耗。
- **观察与中止：**记录关键动作、工具结果和测试信号，并允许工程师暂停、接管或终止任务。
- **结果提升：**只有通过验证的修改集才能进入共享分支或部署环境，运行环境本身不授予交付资格。

演讲以“为了让代理继续工作而保持笔记本打开”的场景说明持续运行痛点，并提出多个任务在各自独立执行单元中并行运行的方向。演讲没有限定容器、虚拟机或其他具体隔离技术，也没有给出隔离强度与安全认证。文件、权限、资源、观察和结果提升是该机制落地时仍需补充的控制面，不能视为演讲者逐项给出的安全保证。

隔离执行环境不自动构成安全保证

工作区分离只能降低一部分任务间干扰。网络出口、身份凭据、供应链依赖、宿主机边界、日志敏感信息和数据驻留仍需单独治理。实际隔离强度应由威胁模型和合规要求决定。

5.5 任务应在哪个环境运行

演讲者把 Codex Cloud 称为此前的首个重要发布，并在会议现场表示它将迎来重要升级。<sup>29</sup>这段话说明会议时点已经存在具名的云端产品入口；“将迎来升级”仍是当时的未来陈述，不能改写为当前已经交付的状态。已有云端入口与下文的自动环境选择是两个层次：前者是会议现场产品快照，后者是统一代理的未来愿景。

本地—云端任务路由（local/cloud routing）指依据数据位置、权限、硬件、持续运行需求、成本与可用性，为任务选择本地或云端执行资源。演讲的未来愿景是由统一代理理解任务、判断适合的环境并使用可用资源，从而减少用户预先选择“本地任务”或“云端任务”的操作负担。自动选择、跨环境回退与数据治理仍属待落实机制。

表 9: 本地—云端任务路由的条件清单

| 判断因素      | 偏向本地执行的信号             | 偏向云端执行的信号                | 必须保留的控制    |
|-----------|-----------------------|--------------------------|------------|
| 数据与权限     | 数据受设备或内网边界约束，工具依赖本地身份 | 已批准的数据可进入托管环境，远程身份可被最小授权 | 数据清单、授权记录  |
| 资源与工具     | 依赖本地设备、专用软件或低延迟交互     | 需要弹性计算、专用加速器或标准化远程工具链    | 能力探测、版本固定  |
| 持续运行      | 任务短、需要频繁亲手介入          | 任务长、需跨设备继续或无人值守执行        | 超时、中止、恢复   |
| 并行与隔离执行环境 | 并发低且本地资源充足            | 多任务需独立环境和弹性扩容            | 配额、日志、结果隔离 |
| 失败回退      | 本地状态是唯一可信来源           | 云端可重建环境并保存可审计产物          | 回退路径、产物校验  |

路由决策应以任务为单位，同一项目中的不同子任务可以落在不同环境。包含敏感数据的分析可留在本地；耗时构建、兼容性测试或多候选探索可在获批的云端环境中运行；最终合并与部署仍按组织的验收和授权链执行。统一入口只能减少操作负担，无法消除权限判断。

<sup>29</sup>原视频：00:46:48–00:47:03。

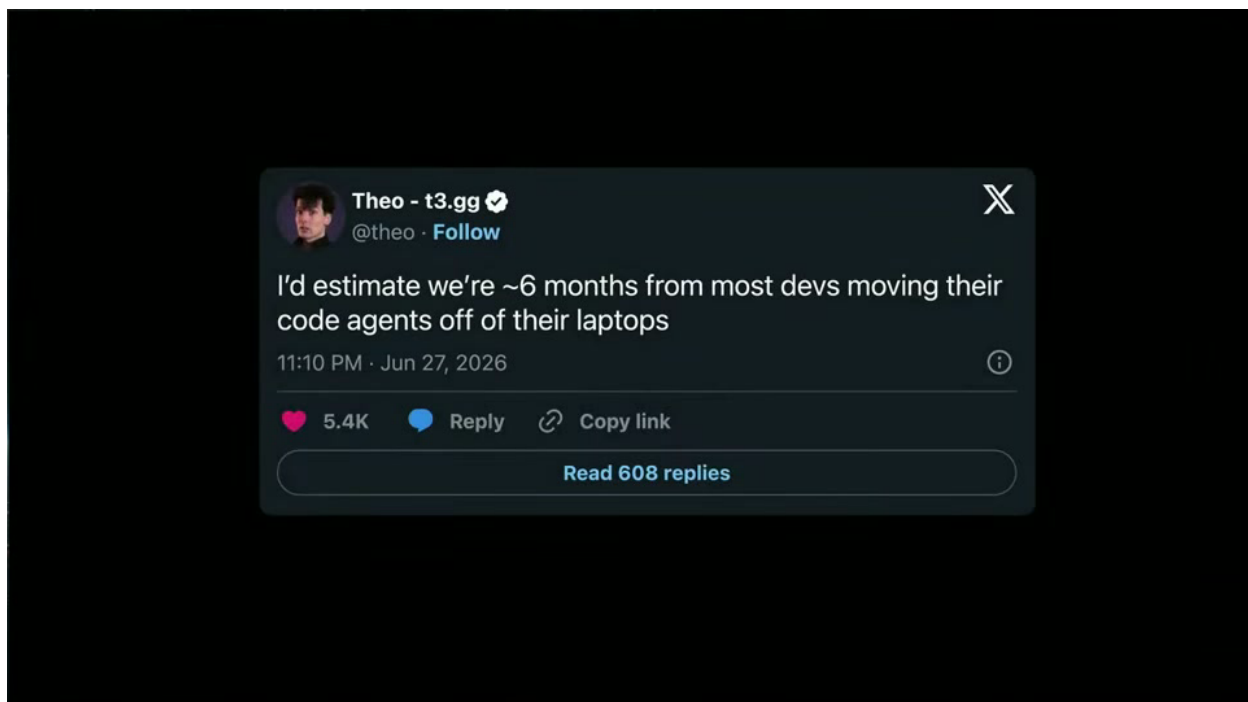


图 13: 外部预测的现场引用: Theo 估计约六个月后, 多数开发者会把代码代理移出笔记本; 演讲者只认同迁移趋势可能更快, 未作发布日期承诺。<sup>30</sup>

## 5.6 本章小结

- 价值评估以通过验收的完整任务为单位, 成本、速度和运行位置共同决定结果能否持续交付。
- 基准得分、价格和 750 tokens/s 测量不同对象, 所有数字均属于会议现场快照, 端到端任务仍需独立测量。
- 隔离执行环境支持长时与多任务并行, 其安全强度取决于权限、网络、凭据、资源和数据治理。
- 本地—云端任务路由应保留人工可见性、停止与回退能力; 自动选择环境属于待验证的未来产品形态。

成本、速度和运行位置给出了价值判断的三个观察面。末章将进一步把这些判断落实为目标、权限、观察、独立验证、回退与责任框架。

<sup>30</sup>原视频: 00:47:21–00:47:36。图中的“约六个月”来自会议现场引用的外部帖子, 属于预测性观点; 演讲者对“可能更早”的回应属于趋势判断。二者均不构成 OpenAI 路线图、交付日期或已经完成的行业迁移事实。自动路由仍依赖数据驻留、权限、环境能力探测、失败回退、成本控制和区域可用性。

6 总结与延伸：工程师掌控下的智能体工作系统

全篇结论：五项落地条件必须共同成立

可持续的智能体工程系统需要清晰目标、受控环境、可观察执行、独立验证和人工责任归属。开放代理栈扩大可组合能力，对话式委派与亲手介入保留工作掌控感，本地—云端任务路由扩展执行资源；最终交付仍由工程师定义成功、限制权限、判断证据并承担结果责任。

前五章形成了一条完整链路：工程实践先定义要解决的问题；任务委派把目标交给能够运行和反馈的智能体；协作式界面让工程师在高层委派与细节介入之间切换；开放代理栈提供模型、接口、智能体运行框架、应用控制与扩展能力；成本、速度和运行位置决定这些能力是否能形成可持续价值。末章把这条链路压缩成可执行的任务设计与验收框架。

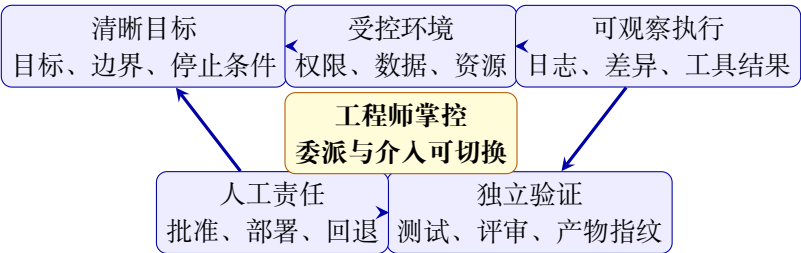


图 14: 工程师掌控下的智能体工作系统（教学重绘）：五项条件形成可回退闭环，任何一项缺失都会削弱交付可信度。<sup>31</sup>

6.1 从目标到验收的落地清单

任务卡应在执行前写清目标、边界、资源和验收。智能体接收的工作单位越完整，隐含假设造成的风险越大；因此，每次任务委派都应留下可检查的输入、运行记录、验证证据和责任人。

表 10: 从目标到验收的任务闭环

| 阶段    | 必须回答的问题               | 应留下的可验收证据               |
|-------|-----------------------|-------------------------|
| 定义目标  | 要解决什么问题，成功与失败分别怎样判断   | 目标陈述、验收条目、明确排除项、责任人     |
| 设定边界  | 可用哪些数据、工具、权限、环境、时间和预算 | 权限清单、数据分类、环境选择、停止条件     |
| 执行与观察 | 智能体正在做什么，失败后如何修正或中止   | 操作日志、修改集、工具结果、构建与测试记录   |
| 独立验证  | 哪些信号能在智能体自述之外证明结果正确   | 可重复测试、静态检查、人工评审、来源与产物指纹 |
| 验收与交付 | 谁批准合并或部署，出现问题怎样回退     | 验收记录、批准人、发布范围、回滚方案、后续监测 |

一份可执行的检查清单应至少覆盖以下问题：

<sup>31</sup>机制依据来自原视频 00:33:36–00:34:05、00:35:58–00:37:10、00:39:06–00:42:24 与 00:46:41–00:47:23；该图是教学重绘，不是原视频中的单一图示。



1. **目标可验证**：输出形式、功能行为、性能边界和完成条件已经明确，主观词已转换为可观察信号。
2. **上下文足够**：任务所需代码、文档、依赖和业务约束可取得，过时信息与缺失信息已经标记。
3. **权限最小化**：文件、命令、网络、凭据和外部服务只开放完成任务所需范围，敏感操作设有人为批准点。
4. **环境受控**：本地或云端位置符合数据边界，隔离执行环境、配额、超时、中止和恢复机制已经设置。
5. **执行可观察**：关键工具调用、修改原因、失败反馈和重试可追踪，工程师能够暂停并亲手介入。
6. **验证相互独立**：构建一测试闭环使用可重复信号，最终判断不依赖同一智能体对自身结果的概括。
7. **成本受约束**：模型调用、并行候选、远程计算、重试和人工评审设有预算与告警。
8. **责任可追溯**：合并、部署、数据变更和回退均有明确批准人，自动执行不会消除责任归属。

#### 验收必须针对结果和影响

“任务已完成”只是执行状态。工程验收还要确认行为符合目标、修改没有越界、测试覆盖关键风险、依赖与数据处理符合约束、部署影响可接受，并且失败时能够回退。

## 6.2 开放代理栈中的责任分配

六层开放代理栈提供不同的参与位置。落地时，每一层都需要明确“谁提供能力、谁配置边界、谁检查结果”。表中责任分配构成实施时的最小治理边界，具体职责仍需由平台方、插件提供方、集成方与最终批准人按组织环境确认；演讲未披露 OpenAI 内部的完整治理制度。

表 11: 开放代理栈的落地责任与验收证据

| 层级            | 主要职责                 | 集成方责任                    | 关键验收证据            |
|---------------|----------------------|--------------------------|-------------------|
| Models        | 提供推理、生成与工具调用能力       | 选择适合任务的能力层级, 评估已知限制与失败模式 | 任务集评测、失败样本、版本记录   |
| Responses API | 暴露模型能力与共用原语          | 管理身份、版本、速率、重试、数据发送和调用来源  | 请求日志、版本固定、错误处理记录  |
| Codex Harness | 承载智能体循环、工具协议、环境访问与状态 | 配置工具允许列表、执行边界、停止条件和状态恢复  | 工具轨迹、环境清单、中止与恢复测试 |
| App Server    | 连接智能体运行框架与多个客户端      | 管理会话、认证、任务状态、并发和可观察性     | 会话审计、权限检查、状态一致性记录 |
| App           | 向用户呈现委派、进展、产物与介入入口   | 设计批准点、风险提示、局部修改和人工接管路径   | 界面状态、批准记录、人工介入记录  |
| Plugins       | 加入工具、知识和专业流程         | 审查来源、权限、输入输出、故障影响和撤销方式   | 插件清单、授权范围、隔离与撤销测试 |

开放源代码、开放接口和扩展点提高了检查、复用与改造能力。它们不会自动提供安全性、兼容性或可靠性保证。集成方仍需验证版本、权限、依赖、数据流和故障边界；平台方、插件提供方与使用组织之间的责任应写入可执行约定。

### 6.3 演讲者的实质性收束

演讲末尾没有继续安排传统现场演示。两位演讲者把这一时刻概括为“重新想象如何工作和构建”（原视频约 00:47:54–00:47:59），随后邀请下一位特殊嘉宾登台。礼貌性交接不承担技术论证，真正的收束来自此前已经建立的主线。

#### 演讲的三条实质结论

- **工作方式需要重构：**智能体进入计划、构建、审查和部署后，工程师把更多精力放在目标、约束、判断、验收和责任承担。
- **开放栈形成反馈回路：**外部开发者可以检查、复用、分叉或扩展部分组件，社区使用经验再影响产品能力与接口设计。
- **价值来自完整任务：**成本、速度和运行环境共同影响交付，统一代理与自动环境选择属于仍需工程治理支撑的方向。

这三条主线也解释了 Codex App 的角色。应用界面提供对话式委派、进度观察、产物检查和亲手介入；开放代理栈让这些交互连接模型、工具、环境与插件；工程师使用验收标准决定何时继续委派、何时接管、何时合并以及何时停止。

### 6.4 开放问题与下一步

#### 开放问题

- 自动路由依据哪些可解释信号选择本地或云端环境，错误选择由谁发现并纠正？
- 隔离执行环境采用何种技术边界，凭据、网络出口、宿主机和供应链风险怎样控制？
- 多候选并行后由谁比较结果，比较器如何避免把测试通过误当作产品目标达成？
- 长时任务中的上下文压缩会丢失哪些约束，关键事实如何被固定、复核和恢复？
- 模型、远程环境和人工评审的总成本怎样归集到一个任务，并与返工率和交付质量共同观察？
- 平台、插件提供方、集成团队和最终批准人之间的事故责任怎样划分？

#### 具体下一步

1. 选择一个范围有限、可回退、已有测试的真实任务作为试点，避免从高权限生产操作开始。
2. 写一页任务卡，列明目标、排除项、输入、输出、验收信号、预算、最长运行时间和责任人。
3. 建立数据与权限矩阵，标明哪些内容可留在本地、哪些内容可进入获批云端环境、哪些操作需要人工批准。
4. 为本地—云端任务路由制定初始规则和人工覆盖入口，并演练网络中断、资源不足和环境误选时的回退。

- 5. 把构建—测试闭环接入可重复验证，再增加独立评审，确保智能体自述不会成为唯一完成证据。
- 6. 记录每次试点的端到端时长、调用与计算成本、重试次数、人工介入、缺陷和回滚情况。
- 7. 根据真实失败样本调整工具权限、隔离执行环境的强度、任务粒度和验收标准，再逐步扩大任务范围与并发量。

6.5 本章小结

- 工程师通过清晰目标、受控环境、可观察执行、独立验证和人工责任归属驾驭智能体。
- 开放代理栈提供可组合能力，每一层都需要明确权限、边界、验收证据和责任主体。
- 会议现场的型号、得分、价格、吞吐量和外部预测只能作为时点证据，真实采用必须重新测量与核验。
- 最稳妥的落地路径是从可回退试点开始，用失败样本逐步改进本地—云端任务路由、隔离执行环境、验证和人工批准机制。

来源说明

证据类型与使用限制

下表集中说明正文采用的四种证据强度；型号、价格、评测与预测都需要保留会议时点和说话者归属。

表 12: 全篇证据类型及其使用方式

| 证据类型      | 可支持的内容                        | 使用限制                     |
|-----------|-------------------------------|--------------------------|
| 源事实       | 字幕明确陈述、界面可见内容、六层结构、工作流阶段、画面数字 | 只覆盖可听见或可看见的范围,不能补写未披露实现  |
| 演讲者判断     | 工程价值、产品交互、生态策略、趋势与经验性比较       | 保留说话者归属,不能扩展为普遍规律        |
| 会议现场快照    | 型号、基准得分、价格、产品状态、750 tokens/s  | 与会议时间和条件绑定,未独立复核,不代表当前规格 |
| 未来愿景或外部预测 | 自动选择本地或云端环境、云端执行趋势、约六个月预测     | 依赖与交付日期未确定,不构成路线图承诺      |

- TerminalBench 2.1 只提供单一评测横截面，无法证明所有仓库、语言、权限环境和长时任务中的总体表现。
- Terra、Luna、Sol 等型号、得分和价格均为会议现场信息，后续可用性与计价条件可能变化。
- 750 tokens/s 衡量生成吞吐量的一部分，完整任务还包含工具、编译、测试、网络、比较和人工验收。
- “约六个月” 来自 Theo 的外部预测；演讲者的回应只表达趋势判断。

- “内部产品与开发者使用同源组件” 的证据范围限于演讲列出的模型接口、Codex Harness、App Server 和扩展点。内部基础设施、权限、安全策略、训练数据和完整部署配置没有被披露。
- 隔离执行环境控制清单、本地—云端任务路由条件和独立验收方法是面向落地的工程控制建议，其适用性取决于组织自身的系统、数据、威胁模型和治理要求；演讲没有提供安全、合规或可靠性保证。
- **原视频标题：** *WF2026: Software Factories & Keynotes ft. Microsoft, OpenAI, OpenClaw, Z.ai (GLM), MiniMax, HF*，发布频道为 AI Engineer。
- **原视频链接：** <https://www.youtube.com/watch?v=htM02KMNZnk>。
- **本文主题区间：** 名义区间为原视频 00:28:37–00:48:09；00:48:09–00:48:12 只用于补全登台交接句，下一主题内容不在本文范围内。
- **文本主证据：** 英文原始字幕用于核对陈述、术语、数字和时间；画面用于核对身份页、系统分层、产品界面、基准图、速度页和外部帖子。
- **演讲者身份：** 原视频身份页清晰显示 Alexander Embiricos 与 Romain Huet，正文采用这两个画面可证实的姓名。
- **适用范围：** 本文可用于设计智能体工作流，无法替代组织自身的威胁建模、隐私评估、合规审查、采购核验和生产变更审批；任何进入真实数据或生产系统的任务都需要按实际环境重新验证。
- **阅读方式：** 正文提供脱离视频也能理解的完整教学解释；原视频只作为来源核对、现场语气与画面补充资料。